Разведочный анализ данных (EDA) и предобработка

Часть 2. Метрики расстояния, скейлинг, мультиколлинеарность, регуляризация и Feature Engineering

Лекция по машинному обучению

Осень 2026

Введение: Как машина начинает «понимать» мир?

Компьютер не обладает человеческой интуицией:

- Он не видит уютную квартиру, красивую фотографию или привлекательную цену.
- Для алгоритма любые данные — это просто безликая матрица из нулей и единиц, таблица из чисел.

Главный вопрос машинного обучения

Как заставить холодную математику находить закономерности, определять похожие объекты и делать правильные выводы?

Реальный мир вокруг нас

- **Рекомендации (Кинопоиск, Яндекс):** откуда система «знает», какой фильм вам предложить?
- **Аренда (Airbnb):** как алгоритм оценивает справедливую стоимость жилья?

В основе всех умных систем лежит одна идея — перевод логики в язык расстояний и геометрии.

Введение: Превращение таблицы в многомерную Вселенную

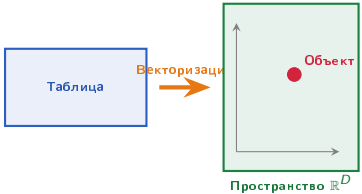
Магия абстракции в Machine Learning:

1. Берем реальный объект (например, квартиру на Airbnb).
2. Описываем его признаками: площадь, цена, число комнат, удаленность от центра.
3. Превращаем обычную строку таблицы в **точку (вектор)** в многомерном пространстве $\mathbb{R}^D$.

Но есть ловушка...

Сырые данные коварны. Если слепо скормить их алгоритму, математика сломается, а модель начнет принимать абсурдные решения из-за банального несоответствия масштабов.

От таблиц к геометрическому миру



Сегодня мы разберем, как навести порядок в этом пространстве.

Введение: Алгоритмы на основе расстояний и их ловушки

Сила геометрического подхода:

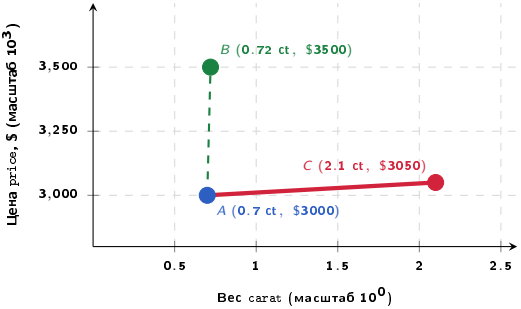
- Опираясь на идею точек в пространстве, люди создали целое семейство **метрических алгоритмов**.
- Самый яркий пример — **kNN** (k-ближайших соседей), о котором мы сегодня поговорим!

Главный подводный камень

Как только мы начинаем измерять расстояния, вылезает масса скрытых проблем (например, разница шкал признаков).

Цель лекции: понять, как с этим бороться и научиться самим реализовывать такие алгоритмические подходы с нуля!

Искажение расстояний из-за масштаба



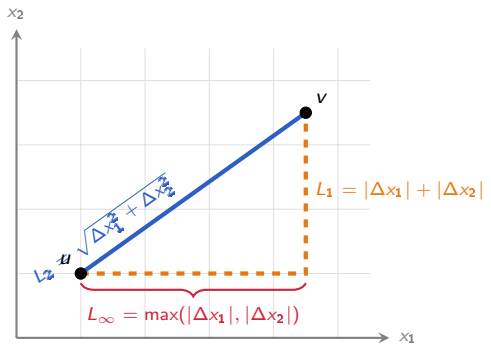
1. Геометрия данных: Метрики расстояния в $\mathbb{R}^D$

Каждый объект описания — это точка (вектор) в D-мерном пространстве признаков: $u, v \in \mathbb{R}^D$.

Классические метрики семейства Минковского (L_p)

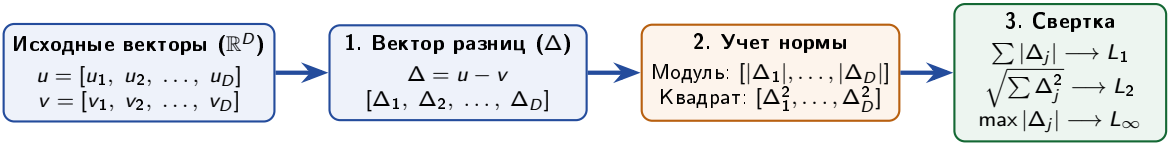
- **Евклидово расстояние (L_2):** кратчайшая прямая $d_{L_2}(u,v) = \sqrt{\sum_{j=1}^{D}(u_j - v_j)^2}$
- **Манхэттенское расстояние (L_1):** движение по сетке осей $d_{L_1}(u,v) = \sum_{j=1}^{D}|u_j - v_j|$
- **Расстояние Чебышёва (L_∞):** максимум по одной оси $d_{L_\infty}(u,v) = \max_{j=1,\ldots,D}|u_j - v_j|$

Геометрическая интерпретация на плоскости



1. Покоординатная механика: Вычитание объектов как векторов

Как вычисляется расстояние «под капотом»: переход от двух строк таблицы к скалярному числу происходит за три элементарных векторных шага.



Роль квадрата в Евклидовой метрике

Возведение разности в квадрат Δ_j^2 не только убирает знак, но и **нелинейно усиливает вклад больших отклонений** по отдельной координате.

Инженерное следствие

Любая метрика расстояния сводится к базовым операциям линейной алгебры над массивами: поэлементному вычитанию и суммированию вдоль оси признаков.

1. Вычислительная эффективность: Цикл `for` против Broadcasting

Задача: найти расстояния от целевого объекта $q \in \mathbb{R}^{1\times D}$ до всех N объектов $X \in \mathbb{R}^{N\times D}$.

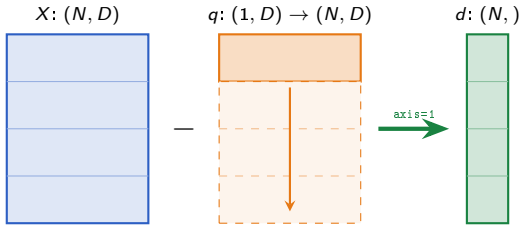
Почему цикл `for` в Python медленный?

- На каждой из N итераций проверяются типы данных и выделяется память.
- Вычисления последовательные на одном потоке процессора.

Секрет скорости NumPy

- Данные в непрерывном блоке памяти C-массива без накладных расходов.
- **Broadcasting** виртуально растягивает вектор $(1, D)$ до размера (N, D) и выполняет вычитание параллельно.

Схема 2D-Broadcasting: $(N, D) - (1, D)$



Суммирование вдоль `axis=1` «схлопывает» ось признаков D, оставляя вектор из N готовых расстояний.

1. Матрица попарных расстояний: 3D-Broadcasting ($N \times N$)

Задача «все со всеми»: найти расстояния между каждой парой из N объектов (N^2 пар).

Трюк с фиктивной осью (`None`)

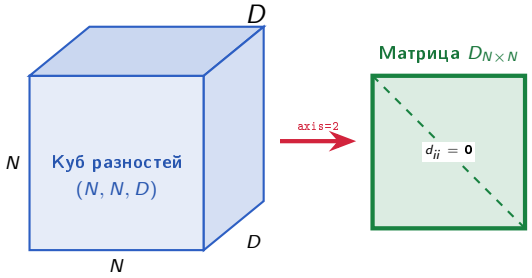
Размерности не совпадают, переводим матрицы в трехмерное пространство:

- $A = X[:, \texttt{None}, :]$ имеет форму $(N, 1, D)$
- $B = X[\texttt{None}, :, :]$ имеет форму $(1, N, D)$
- Разность $C = A - B$ растягивается в 3D-куб (N, N, D), где $C_{i,j,k} = X_{i,k} - X_{j,k}$.

Память vs Скорость

Сложность по памяти $\mathcal{O}(N^2 D)$: при больших N куб разностей создается батчами, чтобы не переполнить RAM.

Растяжение в куб (N, N, D) и свертка по



2. Два главных метода линейного масштабирования

Чтобы уравнять признаки в правах, перед расчетом расстояний их приводят к единому безразмерному масштабу.

1. Min-Max нормализация (`MinMaxScaler`)

$$x_{\text{scaled}} = \frac{x - x_{\min}}{x_{\max} - x_{\min}}$$

- Сжимает все значения признака строго в отрезок $[0; 1]$.
- **Главный минус:** крайняя чувствительность к выбросам. Один аномальный объект $x_{\max}$ сожмет всю основную выборку в узкую полоску около нуля.

2. Z-score стандартизация (`StandardScaler`)

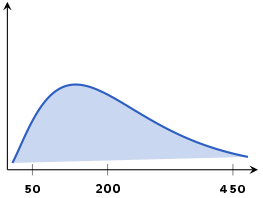
$$z = \frac{x - \bar{x}}{\sigma}$$

- Сдвигает среднее в 0 ($\bar{z} = 0$) и делает единицей измерения стандартное отклонение ($\sigma_z = 1$).
- **Главный плюс:** устойчивее к выбросам и является стандартом де-факто для линейных и метрических алгоритмов.

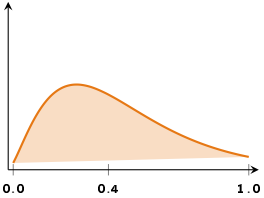
2. Фундаментальное свойство: Скейлинг НЕ меняет форму распределения

И Min-Max, и Z-score являются **линейными преобразованиями** вида $x_{\text{new}} = a \cdot x + b$ (при $a > 0$).

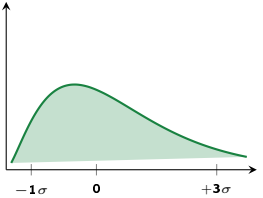
1. Исходный признак (x)



2. Min-Max ($[0; 1]$)



3. Z-score ($\mu = 0, \sigma = 1$)



Главный теоретический вывод

- При линейном масштабировании **меняется только подпись шкалы по оси** X (сдвиг начала координат и масштаб сетки).
- Коэффициент асимметрии (Skewness), относительное положение среднего/медианы и наличие выбросов остаются **абсолютно неизменными**! (Для исправления скоса нужны нелинейные операции, например ln).

3. Восстановление пропусков: Векторизованный алгоритм kNN

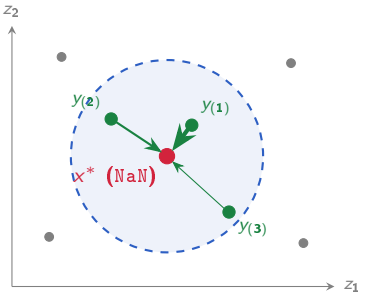
Принцип сходства: близкие в пространстве объекты имеют близкие значения целевого признака.

Пошаговая математика взвешенного kNN

Для каждого объекта с пропуском x^*:

1. Вычисляем матрицу квадратов расстояний до всех N объектов через 3D-Broadcasting: $(M, 1, D) - (1, N, D)$.
2. Находим индексы k ближайших соседей с помощью частичной сортировки (`argpartition`).
3. Вычисляем веса, обратно пропорциональные расстоянию: $w_i = \frac{1}{d(x^*, x_{(i)}) + \varepsilon}, \quad \varepsilon = 10^{-5}$
4. Итоговая оценка — взвешенное среднее: $\hat{y}^* = \frac{\sum_{i=1}^{k} w_i y_{(i)}}{\sum_{i=1}^{k} w_i}$

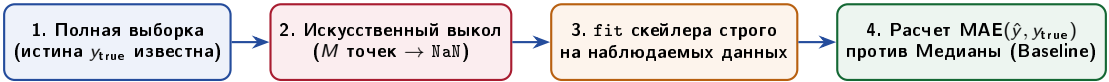
Поиск соседей и взвешивание ($k = 3$)



Чем ближе сосед, тем толще стрелка (больше вес w_i).

3. Валидация импутации и критический эффект масштабирования

Как честно измерить качество заполнения пропусков, если истинные значения неизвестны?



Поведение kNN БЕЗ масштабирования

Если среди признаков есть величина с огромным разбросом (например, стоимость в тысячах), метрика расстояния подбирает соседей **исключительно по ней**, полностью игнорируя точные физические габариты объекта. Ошибка возрастает в разы!

Поведение kNN СО стандартизацией

После приведения всех признаков к $\sigma = 1$ физически связанные координаты получают равный вес в пространстве расстояний. Алгоритм находит истинно похожие объекты и на порядок превосходит наивную медиану.

3. Что будет, если обучить модель на «сырых» координатах?

Посмотрим на реальный пример из нашего датасета: возьмем вес алмаза `carat` (0.2 . . . 5.0) и цену `price` ($300 . . . $18 000) и отправим их в метрическую модель без обработки.

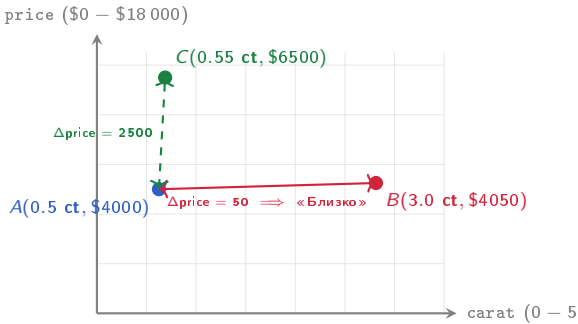
В чем проблема на графике справа?

- По оси X (`carat`) разница всего $\Delta x_1 = 2.0$.
- По оси Y (`price`) разница $\Delta x_2 = 5\,000$.
- Для формулы расстояния разница в 2 карата **ничтожна** на фоне тысяч $$$!

Последствие для обучения

Модель посчитает два совершенно разных по размеру алмаза «близнецами», если их цена случайно совпала. Качество предсказаний упадет!

Взгляд на данные: Иллюзия близости



3. Осторожно, синтетика! Опасности восстановления данных в продакшене

Любое восстановление пропусков создает **искусственное значение**, которое полностью зависит от текущего состояния датасета и не имеет реальной физической природы.

Риск «галлюцинаций» модели

- **Нереалистичные комбинации:** импутация может породить вектор с полями, которые никогда не встречаются вместе в живой природе (даже после стандартизации!).
- **Критичные домены:** в медицине или инфобезопасности такие «франкенштейны» в обучающей выборке заставят модель галлюцинировать на реальных пациентах или атаках.

Строгие правила применения

1. **Только числовые признаки:** мы восстанавливаем числа там, где физически понимаем все поля и можем глазами проверить адекватность комбинации (например, габариты камня).
2. **Табу на сложный тип** `object`**:** сырые тексты и сложные структуры требуют контроля векторизации и трансформеров — до их изучения текстовые пропуски закрываем только базовой заглушкой (`"Unknown"`).

Коэффициент линейной корреляции Пирсона (r)

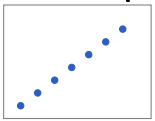
$$r_{xy} = \frac{\sum_{i=1}^{n}(x_i-\bar{x})(y_i-\bar{y})}{\sqrt{\sum_{i=1}^{n}(x_i-\bar{x})^2 \sum_{i=1}^{n}(y_i-\bar{y})^2}} = \frac{\mathrm{Cov}(x,y)}{\sigma_x \sigma_y}$$

- Принимает значения строго в отрезке $[-1; 1]$.
- Измеряет силу и направление **линейной** связи.

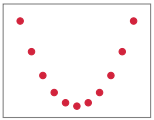
Две цели анализа тепловой карты (Heatmap)

1. **Отбор предикторов:** поиск признаков с высоким $|r(x_j, y)|$ по отношению к таргету.
2. **Детекция мультиколлинеарности:** выявление блоков признаков, дублирующих друг друга ($|r(x_j, x_k)| > 0.8$).

Линейность против Нелинейности

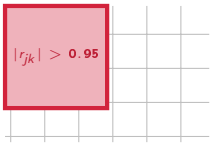


$r \approx +0.98$



$y = x^2 \implies r = 0!$

Очаг мультиколлинеарности в матрице R:



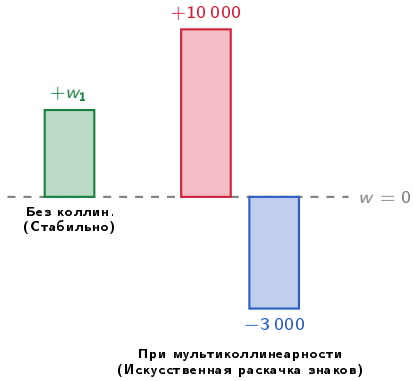
4. Проблема мультиколлинеарности в линейных моделях

Аналитическое решение метода наименьших квадратов (OLS): $\hat{w} = (X^T X)^{-1} X^T y$.
При наличии линейно зависимых признаков матрица $X^T X$ становится почти вырожденной: $\det(X^T X) \to 0$.

Три разрушительных последствия

1. **Взрыв дисперсии весов:** при смене подвыборки или добавлении микрошума 10^{-5} коэффициенты w_j скачут на тысячи единиц.
2. **Потеря физического смысла (инверсия знаков):** модель раздувает вес одного признака в огромный плюс, а дублирующего признака — в огромный минус («чем больше размер объекта, тем он дешевле»).
3. **Хрупкость:** любая погрешность в одной из фичей разрушает баланс и ломает прогноз.

Эффект «перетягивания каната»



весами

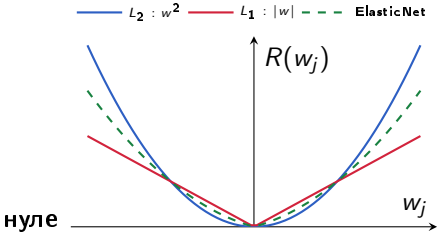
5. Регуляризация: Укрощение весов и отбор признаков

Добавляем к функции потерь штраф за величину коэффициентов (требует обязательного `StandardScaler` перед обучением!):

$$L_{\text{reg}}(w) = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2 + \alpha \cdot R(w)$$

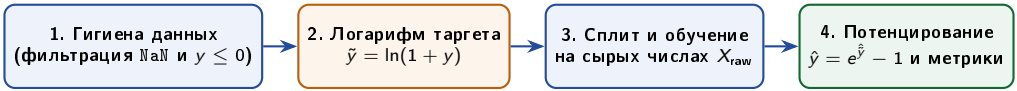
Метод	Штраф $\alpha R(w)$	Воздействие на веса w_j
Ridge (L_2)	$\alpha \sum w_j^2$	Плавно сжимает веса к нулю, подавляя разброс, но **не обнуляет** их строго.
Lasso (L_1)	$\alpha \sum \lvert w_j \rvert$	За счет излома модуля в нуле **строго зануляет** избыточные и дублирующие признаки (Feature Selection).
ElasticNet	$\alpha(p\lVert w\rVert_1 + (1-p)\lVert w\rVert_2^2)$	**Эффект группировки:** выкидывает шум, а группе скоррелированных фичей дает близкие физически верные веса.

Профиль штрафа $R(w)$ и излом L_1 в нуле



6. Feature Engineering: Точка отсчета (Baseline Pipeline)

Прежде чем генерировать сложные признаки, необходимо зафиксировать **Baseline** — базовую модель на сырых очищенных числовых столбцах.



Почему сырые признаки работают слабо?

Линейная модель ищет гиперплоскость $\hat{y} = w_0 + \sum w_j x_j$. Она не способна сама вычислить радиальное расстояние из географических координат или извлечь смысл из текста и дат.

Двойной контроль качества

Оцениваем модель одновременно по R^2 / $\mathbf{MAE}_{\log}$ (качество математической оптимизации) и по **MAE в реальных единицах** после обратного преобразования $\text{expm1}(\hat{\tilde{y}})$ (понятная бизнесу ошибка).

6. Конструирование признаков: Перевод знаний о мире в числа

Философия Feature Engineering: явно подать линейной модели нелинейные закономерности предметной области.

Источник данных	Математическое преобразование	Доменная логика для линейной модели
1. Гео-координаты (ϕ, λ)	$d_{\text{center}} = \sqrt{(\phi - \phi_0)^2 + (\lambda - \lambda_0)^2}$	Сырые широта и долгота задают плоскость, а радиус до центра напрямую отражает падение цены к окраинам.
2. Даты событий (t_{event})	$\Delta t = t_{\text{snapshot}} - t_{\text{event}}$ (в днях)	Перевод календарной строки в «давность активности»: свежая активность сигнализирует о высоком спросе.
3. Сырой текст заголовка	$\mathbb{I}[\text{regex_match(text)}]$	Бинарный индикатор наличия маркеров премиум-сегмента или ключевых характеристик.
4. Счетчики активности	$\mathbb{I}[\text{count} > 1]$	Пороговая бинаризация: отделение частных лиц от профессиональных коммерческих игроков.
5. Ранговые категории	Map : $C_1 \to 1,\ C_2 \to 2,\ C_3 \to 3$	**Ordinal Encoding:** сохранение естественного монотонного порядка возрастания ценности/комфорта.

6. Категориальные признаки малой мощности: One-Hot Encoding

Выбор метода кодирования номинальных категорий (без естественного порядка) определяется их **мощностью** K (числом уникальных значений).

Когда мало категорий ($K \leq 10$) → One-Hot (OHE)

Каждая категория превращается в отдельный бинарный столбец-флаг $\mathbb{I}[x = c_k] \in \{0, 1\}$.

Dummy-ловушка и как её избежать

- Сумма всех K бинарных столбцов в каждой строке всегда равна 1 (полная коллинеарность со свободным членом w_0).
- **Правило:** всегда отбрасываем первую категорию (`drop='first'`), оставляя $K - 1$ столбцов!

Схема OHE с `drop='first'`

Округ ($K = 3$)	is_Manhattan	is_Brooklyn
Bronx *(база)*	0	0
Manhattan	1	0
Brooklyn	0	1

Если оба флага равны 0, модель однозначно понимает, что это базовый округ `Bronx`.

6. Категории высокой мощности ($K \gg 20$): Target Encoding

Если уникальных значений сотни (например, 221 микрорайон города), OHE создаст сотни пустых столбцов и раздует размерность пространства D.

Решение: Сглаженный Target Encoding

Упаковывает сотни категорий в **один числовой столбец**, заменяет на сглаженное среднее таргета в ней:

$$\mathrm{TE}(c) = \frac{n_c \cdot \bar{y}_c + m \cdot \bar{y}_{\text{global}}}{n_c + m}$$

- $n_c, \bar{y}_c$ — число объектов и среднее таргета в категории c.
- $\bar{y}_{\text{global}}$ — глобальное среднее по всему `Train`.
- m — вес сглаживания (защищает от переобучения на редких категориях из 1–2 строк).

Защита от утечки (Data Leakage)

Target Encoding напрямую использует целевую переменную y!

- Вычисляется **строго после сплита** только на `Train`.
- Внутри `Train` используется кросс-валидация (`Out-of-Fold`), чтобы модель не подглядывала в ответ самой строки.

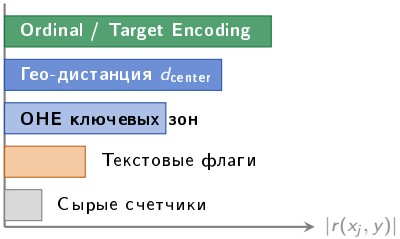
6. Валидация гипотез: Корреляционный аудит новых признаков

После генерации числовых, бинарных и закодированных признаков матрица X проходит повторный корреляционный контроль.

Что мы проверяем на тепловой карте после FE?

1. **Проверка гипотез (связь с таргетом y):** какие из созданных нами признаков дали максимальный линейный отклик $|r(x_j, y)|$? (Именно порядковые ранги, радиальные расстояния и Target Encoding обычно становятся лидерами).
2. **Контроль индуцированной коллинеарности:** не создали ли мы дубликаты? (Например, расстояние до центра города неизбежно коррелирует с бинарным флагом центрального района).

Иерархия силы созданных признаков



6. Feature Importance: Как понять, какие признаки реально важны?

Ловушка сырых весов w_j

Сравнивать сырые коэффициенты линейной регрессии w_j напрямую **нельзя**! Признак, измеряемый в долях градуса, получит огромный вес w_j, а признак в сотнях дней — крошечный, даже если их реальное влияние одинаково.

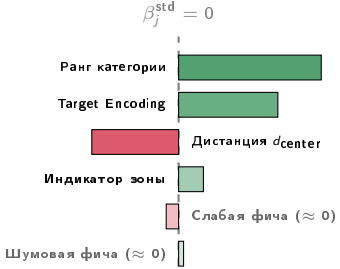
Стандартизированный вклад признака (β_j^{std})

Умножаем сырой вес на стандартное отклонение признака:

$$\beta_j^{\text{std}} = w_j \cdot \sigma_{x_j}$$

Смысл: на сколько изменится таргет $\hat{y}$ при изменении признака x_j на **одно типичное отклонение** ($1\sigma_{x_j}$).

Принцип Парето в Feature Engineering



Не все придуманные фичи одинаково полезны: слабые признаки с $|\beta_j^{\text{std}}| \approx 0$ можно смело отсекать.

Спасибо за внимание!

Вопросы • Обсуждение • Практика

ИТОГИ СЕГОДНЯШНЕЙ ЛЕКЦИИ

Мы прошли полный цикл инженерной работы с признаками:

- **Векторизация и скейлинг:** ускорили расчет расстояний в сотни раз через Broadcasting и спасли метрические алгоритмы от слепоты масштаба.
- **Укрощение линейных моделей:** разобрали природу мультиколлинеарности и научились стабилизировать веса через L_1, L_2 и ElasticNet.
- **Feature Engineering и интерпретация:** кратно подняли точность модели доменными признаками, OHE и Target Encoding, оценив их честный вклад через β_j^{std}.